

Activity 08: NAT DHCP DNS and time

TGS-2023037854 | v7.0 | 13 September 2026 | 100 minutes

Goal

Correlate translated transport tuples, DHCP scope options, DNS records and synchronized logging.

Before you start

Python 3.9+, a plain text editor and this complete folder. Offline mode requires no extra packages, Internet or credentials. The compulsory core Packet Tracer construction is specified below and in topology.md; advanced extensions may need CML/IOS XE.

Evidence boundary

All JSON/CSV observations are synthetic training evidence. Passing the Python checker validates the fixture contracts only. No hardware, simulator, authentication server or API is configured or contacted.

Files

README.md, instructions.pdf, checklist.pdf, topology.md, case.json, faulty.json, reference.json, observations.csv, specimen.cfg, transfer-copy.cfg, S1-baseline.cfg, S2-baseline.cfg, R1-baseline.cfg, R2-baseline.cfg

Working method

case.json is the working evidence; faulty.json resets the exercise and reference.json shows corrected synthetic evidence for self-study. Preserve your own sandbox observations separately.

Detailed step-by-step instructions

1. Construct and save the mandatory simulator topology

Start Cisco Packet Tracer 8.x or a current compatible version. File > New; use the bottom device palette, drag these models into the workspace and rename them via Config > Display Name: 2 x Router2911 (R1,R2), 2 x Switch2960 (S1,S2), 1 x PC-PT (PC1), 1 x Server-PT (SRV). Choose Connections > Copper Straight-Through for the table unless otherwise stated. Switch-to-switch links use Copper Cross-Over or Automatically Choose Connection Type. Click the first device and select its named interface; click the peer and select its named interface. Wait for links to initialize. Save as activity-start.pkt.

Device/port A	Device/port B	Cable
PC1 FastEthernet0	S1 FastEthernet0/1	Straight-through
S1 FastEthernet0/24	R1 GigabitEthernet0/0	Straight-through
R1 GigabitEthernet0/1	R2 GigabitEthernet0/1	Cross-over/Auto
R2 GigabitEthernet0/0	S2 FastEthernet0/24	Straight-through
S2 FastEthernet0/1	SRV FastEthernet0	Straight-through

Save both activity-start.pkt and a working activity-working.pkt; use File > Save As.

2. Assign endpoint addresses and inspect interface identifiers

Set Desktop > IP Configuration > Static on each endpoint.

Endpoint	Address	Mask	Gateway	DNS
PC1	DHCP	DHCP	DHCP option	DHCP option
SRV	192.0.2.50	255.255.255.0	192.0.2.1	192.0.2.50

SRV Services > HTTP: HTTP On and HTTPS On if offered. DNS On; add A record app.example.test ->192.0.2.50. NTP On if available (no authentication for this isolated exercise). PC1 gets DHCP from R1; do not enable a competing SRV DHCP scope.

For routers/switches open CLI. Answer no to the initial configuration dialog and press Enter. Use the exact interface names below; a 2911 uses GigabitEthernet0/0 through 0/2, a 2960/3560 uses FastEthernet0/1 through 0/24 and GigabitEthernet0/1 through 0/2.

```
enable
show ip interface brief
```

3. Install all peer baseline configurations

Paste the following device-specific blocks into each named device CLI one device at a time. Wait for the router/switch prompt between blocks. These are complete activity configurations for the stated construction. Do not apply configuration from a different activity. When copy asks for Destination filename, press Enter. Save the Packet Tracer working file after all blocks.

Device S1

```
enable
configure terminal
hostname S1
no ip domain-lookup
interface range FastEthernet0/1, FastEthernet0/24
  switchport mode access
  no shutdown
exit
end
```

```
copy running-config startup-config
```

Device S2

```
enable
configure terminal
hostname S2
no ip domain-lookup
interface range FastEthernet0/1, FastEthernet0/24
    switchport mode access
    no shutdown
exit
end
copy running-config startup-config
```

Device R1

```
enable
configure terminal
hostname R1
no ip domain-lookup
interface GigabitEthernet0/0
    ip address 10.10.10.1 255.255.255.0
    no shutdown
exit
interface GigabitEthernet0/1
    ip address 10.0.0.1 255.255.255.252
    no shutdown
exit
interface GigabitEthernet0/0
    ip nat inside
exit
interface GigabitEthernet0/1
    ip nat outside
exit
ip dhcp excluded-address 10.10.10.1 10.10.10.20
ip dhcp pool USERS
    network 10.10.10.0 255.255.255.0
    default-router 10.10.10.1
    dns-server 192.0.2.50
exit
access-list 1 permit 10.10.10.0 0.0.0.255
ip nat inside source list 1 interface GigabitEthernet0/1 overload
ip route 0.0.0.0 0.0.0.0 10.0.0.2
ntp server 192.0.2.50
end
copy running-config startup-config
```

Device R2

```
enable
configure terminal
hostname R2
no ip domain-lookup
interface GigabitEthernet0/0
    ip address 192.0.2.1 255.255.255.0
    no shutdown
exit
interface GigabitEthernet0/1
    ip address 10.0.0.2 255.255.255.252
    no shutdown
exit
ip route 10.10.10.0 255.255.255.0 10.0.0.1
end
copy running-config startup-config

show running-config
show ip interface brief
copy running-config startup-config
```

4. Prove the healthy baseline with actual packets

Run every test below before introducing the fault. PC/Desktop > Command Prompt provides ping, ipconfig and arp. Use Simulation mode and Edit Filters to keep ARP, ICMP and the protocol under study; click Add Simple PDU for a ping and Capture/Forward to inspect hops. The first ping may lose one packet during ARP; repeat and record the second result. Do not mark a test passed until observed.

PC1 ipconfig shows 10.10.10.21+ /24 gateway 10.10.10.1 DNS 192.0.2.50. PC1 ping 192.0.2.50 succeeds.

Browser http://app.example.test returns the PT server page. R1 show ip dhcp binding shows the leased client; show ip nat translations during traffic shows inside local 10.10.10.x mapped to inside global 10.0.0.1; show ntp associations/status should show synchronized peer if the PT NTP model supports it. In Simulation inspect DHCP DORA, DNS question/answer and NAT address rewrite.

After all healthy tests pass, File > Save As > activity-start.pkt to preserve the CONFIGURED healthy baseline, then File > Save As > activity-working.pkt before introducing the fault. The initial empty construction is not the rollback baseline.

Save baseline screenshots / CLI text with device names; File > Save.

5. Introduce and diagnose the controlled fault

Save the baseline first. Apply only the following fault to the specified device. Repeat the same traffic tests, capture the first failing hop/control and compare show output with the healthy baseline.

On R1 DHCP pool USERS change dns-server 192.0.2.99. PC1 Desktop > IP Configuration select Static then DHCP to request a new lease. PC1 ping 192.0.2.50 succeeds but Web Browser http://app.example.test fails. Preserve that IP-versus-name difference. Restore DNS then remove R1 ip nat inside from Gi0/0 and inspect missing translations during fresh HTTP sessions; return routing may permit traffic without NAT in this private sandbox, so NAT failure is proven by translation absence, not by assuming ping always fails.

Retain activity-start.pkt and baseline running configurations; work in activity-working.pkt.

6. Apply the narrow repair and re-test the network

Restore R1 pool DNS to 192.0.2.50 and renew the lease. Restore ip nat inside on Gi0/0. Create new traffic and inspect the translation tuple rather than relying on old entries.

Repeat the baseline test matrix. Check surrounding controls as well as the repaired flow. why a successful ping by address cannot prove DNS, HTTP, synchronized time or NAT translation

Save actual show output and allowed/denied traffic results; copy running-config startup-config only after successful verification.

7. Rollback and rehearse recovery

Save the repaired file as activity-verified.pkt. To undo the exercise, open the saved healthy baseline activity-start.pkt after you have saved configurations into it at Step 3, or paste the saved pre-fault running configuration into a clean topology. For the specific repair, retain the exact old command/value rather than guessing a trunk list or ACL. Repeat one positive and one relevant negative test after recovery.

File > Save As > activity-verified.pkt; close working topology; reopen saved healthy baseline.

8. Submit observed evidence and explain the forwarding mechanism

Submit the verified .pkt file, a cabling/address table, pre-fault and post-repair show outputs, screenshots of traffic tests, diagnosis and rollback notes. Explain why a successful ping by address cannot prove DNS, HTTP, synchronized time or NAT translation. Keep synthetic checker output separate from real simulator observations.

Record simulator version, device models and tests actually completed.

9. Complete the separately labelled extension

DNS MX/NS/PTR and SFTP/SCP are transition extensions: PT DNS often supports only A/CNAME. Use the supplied record fixtures for MX preference and reverse names. checker.py calculates a genuine local SHA-256 of transfer-copy.cfg; alter one character and observe rejection, then restore. A real SCP/SFTP transfer requires SSH-capable IOS XE/CML plus a trainer-owned file server; no real transfer is claimed by the local hash test.

Record supported platform, output observed or evidence-only limitation.

Untimed appendix: independent fixture analysis

These synthetic cases supplement the allocated hands-on time and do not configure the simulator.

Appendix 1. Establish the baseline

Open activity08-nat-dhcp-dns-time; inspect topology.md, case.json and observations.csv. Identify the controls for: Correlate translated transport tuples, DHCP scope options, DNS records and synchronized logging.

```
python3 --version
```

Appendix 2. Run the faulty evidence case

Run the checker against case.json; exit status 1 is expected at this stage. Record each FAIL label and the calculated detail.

```
python3 checker.py case.json
```

Appendix 3. Propose the repair

Make translated ports unique by setting the second global_port to 30002; restore DHCP gateway 10.10.10.1; set synchronized true and offset_ms 2. Verify the raw translation fields before concluding NAT succeeds.

Open case.json in a plain text editor; preserve JSON types.

Appendix 4. Validate the corrected case

Save a separate working copy first. Run the checker after each narrow edit until all controls PASS; capture output as evidence. This validates fixtures only.

```
python3 checker.py case.json > evidence.txt
```

Appendix 5. Verify checker behavior

Run self-test; it must accept corrected reference evidence and reject the faulty fixture and the independent corrupted-control test.

```
python3 checker.py --self-test
```

Expected results and evidence

Corrected reference: all checks PASS, exit 0; faulty fixture: one or more FAIL, exit 1; self-test: PASS, exit 0.

Domain-specific computed fields appear in JSON detail.

Capture evidence.txt, corrected case.json and your diagnosis. For sandbox work record device/model, timestamps, actual show outputs, allowed/denied tests and rollback. Distinguish fixture results from real observations.

Troubleshooting and reset

Exit 1: inspect the named failed contract. Exit 2: restore valid JSON keys/types. Unsupported simulator commands must be documented; never invent results. Reset by copying faulty.json over case.json. Restore sandbox settings from the actual saved baseline.

Calculated corrected-fixture results

PAT_global_tuples: [["203.0.113.2", 30001, "tcp"], ["203.0.113.2", 30002, "tcp"]]
DHCP_network: "10.10.10.0/24"
DNS_record_types: ["A", "AAAA", "MX", "NS", "PTR"]
offset_ms: 2

Required PASS controls

- PAT tuple disambiguation
- DHCP scope and options
- DNS forward and refresh records
- NTP selected synchronized source
- secure transfer integrity

Transition practice

Refresh extension: inspect MX preference, NS authority and reverse PTR names; compare SSH-based SCP/SFTP with unauthenticated TFTP. The checker calculates the real SHA-256 of transfer-copy.cfg and compares it with the fixture digests; this proves local file integrity only, not an actual network transfer.

Configuration files

The complete per-device configurations are shown in the construction steps and included as named *-baseline.cfg files. specimen.cfg combines those same blocks with explicit device boundaries. Apply each block only to its named device. WRT300N uses GUI settings instead of IOS CLI.

References

Cisco CCNA v1.1 exam topics; Cisco IOS XE configuration guides; Python ipaddress documentation. Exact URLs and source links are in README.md.